Understanding the evolution of the LoRa APRS project

How the code works, what it provides for operators, and its current limits

English edition, 15 October 2026. iGate firmware reference: `rxt` branch, commit `2cebfe6` dated 13 October 2026. Graywolf is a separate application, examined in the associated `graywolf-rxt` repository.

1. What changes relative to the CA2RXU firmware

Ricardo Guzman’s CA2RXU firmware already provides LoRa communication, APRS digipeating, APRS-IS gating, TNC interfaces and the WebUI. This project retains that base and extends it in two directions: describing what happens on the radio more clearly, and preserving information more faithfully when exchanging data with applications.

An APRS packet identifies its source, requested path and content. Its path can subsequently show which relays were used. On its own, however, it does not tell us the signal level at which each relay received the packet, or how long that relay waited before retransmitting it. RXT, whose concept comes from Jon Adams, N7UV, adds those observations. The JSON stream then makes them available alongside the packet in Graywolf.

The work also covers relay behavior: path handling, limiting repeated forwarding, learning a return path for messages, and local replies. An experimental binary radio transport is an additional development intended to reduce the size of some packets while retaining their exact content.

These functions are distributed across several programs. The iGate measures, relays and exports. Graywolf preserves receptions, decodes APRS content it recognizes, and displays stations and links. The TNC2/JSON path retains extended identities such as `F4MLV-MC` or `F4JJE-16`. The iGate’s current validator also accepts these suffixes for APRS-IS and digipeating, subject to the other processing rules. The limitation concerns conversion to AX.25 on the KISS interface: those suffixes cannot be represented faithfully there.

2. The packet, its identity and its interfaces

TNC2 text as a packet representation

Here is an illustrative packet constructed for this explanation:

```
F4MLV-7>APRS,WIDE1-1:>Bonjour LoRa
```

`F4MLV-7` is the source. `APRS` is the packet destination. `WIDE1-1` requests forwarding according to that alias's rules. The colon ends the header. Here, the information field begins with `>`, identifying a status report.

After a digi forwards the packet, the path changes to identify the relay used. This path change must remain distinct from a change to the message itself. The recipient must be able to recover the original APRS content, with reception measurements providing additional context.

The TNC2 header is textual, but the information field can contain binary bytes, particularly with Mic-E. A readable display is therefore not always a complete copy of the received data. This is why the JSON stream includes a Base64 copy: it carries the bytes without losing them in character conversion.

Why exact suffixes matter

A textual model can retain `F4JJE-16` or `NN7LE-GS` exactly as received. Immediately converting these into conventional AX.25 addresses would create a problem: the AX.25 SSID provides only values 0 through 15 and cannot represent an alphabetic suffix. A permissive conversion could change the station's identity.

Graywolf retains the original address text and checks exact representability whenever an output requires AX.25. It refuses an unrepresentable identity at that conversion. The identity can nevertheless remain in the log, TNC2 model and JSON data.

At this AX.25 boundary, Graywolf rejects an identity it cannot represent exactly. The iGate's historical KISS encoder converts the suffix numerically and does not perform the same strict check, so it can alter the identity. Direct TNC2 preserves extended identities between Graywolf and the radio without that conversion. The iGate's textual validator accepts one- or two-character alphanumeric suffixes, including `-MC`, `-GS` and `-16`. The other APRS-IS admission and digipeater rules still apply.

Three distinct interfaces

Direct TNC2 exchanges text records between the application and the TNC over TCP or serial. **KISS** carries binary AX.25 frames for software and equipment that require them. The **JSON stream** describes receptions and their measurements for an application.

Direct TNC2 avoids a round trip through AX.25 when both ends already understand text. In the iGate, `tnc.kissProtocol` selects KISS or TNC2 mode for the serial and TCP interfaces. In Graywolf, direct TNC2 also sends messages, retries and beacons on an explicitly authorized channel. Receiving JSON or opening a receive connection does not itself authorize transmission.

3. How RXT makes receptions observable

A measurement belongs to the receiver that made it

Consider the historical path described by Jon: CAVECR transmits, SOMTNX relays, then TSRXBX receives. The RSSI measured by SOMTNX describes the link from CAVECR to SOMTNX. TSRXBX's local RSSI describes the next link. These observations concern different receivers and different hops.

SOMTNX can carry its measurement with the packet without having Internet access. The final receiver obtains observations from earlier relays and adds its own local reception when exporting the data. Graywolf can then show directed links rather than assign one quality value to the entire route.

RXT carries four values per relay: received signal level RSSI in dBm, signal-to-noise ratio SNR in dB, frequency offset FO in Hz, and time to hop TTH in milliseconds. TTH runs from the end of reception to the end of the relay's transmission. In the code, it includes queue dwell time, channel-access waiting and calculated airtime for the transmitted frame. It therefore measures more than processor execution time.

Measurements are quantized to fit into a small number of characters. The current code represents SNR from -24 to +20 dB in 0.5 dB steps. TTH encoding depends on the LoRa profile, so decoding requires compatible parameters. A historical value capped at +12 dB must not be interpreted as a precise measurement from the new scale.

Base-89 for RXT, Base64 for JSON

The code calls the compact RXT encoding **Base-89**. Each measurement becomes an index, then an ASCII character obtained by adding 33 to that index. The tuple is not encoded as one block in the way a Base64 string would be: each character represents a separate field. For example, an SNR of 0 dB gives index 48 on the current scale, hence ASCII character 81, `Q`.

The Base-89 name needs a qualification for this implementation. The v2 identifier uses exactly 89 values, calculated as `33 + (fingerprint modulo 89)`, corresponding to characters `!` through `y`. SNR also uses indexes 0 through 88. However, the RSSI, FO and TTH encoders allow indexes 0 through 89 inclusive, reaching character `z` and therefore allowing 90 possible values. Despite its name, the current code does not use a uniform 89-symbol alphabet for every field.

Base64 in JSON serves a different purpose: carrying APRS packet bytes losslessly in a JSON document, including binary Mic-E data. It does not describe the encoding of RXT measurements over the radio. Finally, **encoded APRS station telemetry** uses **Base91** fields, notably for activity rates. These three encodings concern different data and interfaces.

RXT encoding references: constants and encoding functions in `src/lora_utils.cpp`, and the `fingerprint` function in `src/rxt_protocol.cpp`.

Assigning measurements to the correct relay

In text RXT v2, a five-character tuple contains an identifier followed by the four measurements. The identifier fingerprints the relay's callsign, including its suffix and excluding the path star. The decoder compares that fingerprint with the names of used relays.

This removes the dependency on a local whitelist when attributing v2 tuples. It also lets the firmware check that a suffix resembling RXT matches the path. A limitation remains: a fingerprint with 89 values can be shared by several names. Path order resolves some cases. If multiple owners remain possible, the decoder reports the ambiguity instead of choosing arbitrarily.

A relay without RXT can appear in the path without providing measurements. Missing data does not mean the hop did not occur. The firmware retains at most three tuples and adds no RXT to APRS messages, ACKs or REJs, including those encapsulated in third-party frames.

Preserving APRS text in text radio mode

Earlier representations appended a visible RXT trailer to the packet. The current text transport puts RXT in a block after a zero byte: the APRS packet precedes that byte, followed by a marker and the measurements.

Firmware that reads the reception as a C string stops at the zero byte. It therefore obtains the APRS packet without the added measurements. This prevents that third-party iGate from sending RXT to APRS-IS as part of the user information field. The tradeoff is that a conventional relay forwarding only the text loses the hidden measurements from earlier hops.

This compatibility depends on the third-party reader's behavior. It does not cover every possible firmware. The code also continues to read recognized visible trailers. In that internal text representation, a comment that happens to resemble a valid trailer can still be mistaken for RXT.

4. From the receiver to Graywolf

Context accompanies each packet

On reception, the iGate captures local measurements before another radio operation can change them. If the digi decides to relay, the packet and its reception context enter the transmit queue together. A delayed packet must retain its own measurements even if another packet arrives in the meantime.

The main loop then dispatches the reception to enabled functions: APRS-IS, digi, JSON, TNC interfaces, MQTT and SD logging, each under its own conditions. Application outputs use the packet with recognized RXT removed. Measurements remain separately accessible on outputs that can describe them.

The RXT enable switch controls the addition of the relay's measurement during an eligible retransmission. Disabling it does not disable RXT reception, decoding, display or cleanup before APRS-IS.

JSON preserves the packet and its observations

The `/api/v1/aprs/stream` endpoint supplies one JSON object per line. A reception event associates Base64 TNC2 bytes with local measurements, radio parameters and decoded RXT hops. Readable text is an optional projection, included only when the characters can be represented unambiguously in that form.

In normal operation, outside ecoMode, the JSON path can also preserve a non-empty CRC-valid reception that APRS functions subsequently reject, such as a malformed envelope or a blacklisted source. Observing a reception therefore does not mean authorizing it for APRS-IS or RF forwarding.

The iGate retains ten events in memory and provides a cursor for resuming after disconnection. If the cursor requests history that is no longer available, a `gap` event reports it. This memory supports short replay, rather than replacing archival storage in the application.

What the operator can read in Graywolf

Graywolf preserves receptions, decodes APRS content it recognizes, and feeds the log and map. RXT measurements describe remote links. The final device's local reception describes the link from the last used relay, or from the source in a direct reception. No local TTH is invented.

A link is drawn only when the necessary positions are known. A missing line on the map can therefore result from an unknown position even when measurements exist. Link-age colors are not a radio-quality scale. Diagnosing a hop requires reading its measurements and timestamp.

5. Changes to relaying and messages

The path remains a routing decision

The digi checks the path and its operating mode before queueing a packet. The changes include configurable regional aliases, bounded hop counts, explicit paths and a guard against returning through the relay itself. A local 25-second window limits repeated forwarding of the same packet, with separate claims for relaying and route learning.

Adding RXT remains subject to available space. If it no longer fits, the firmware can relay the packet without a new tuple. If the APRS packet itself exceeds the allowed limit, transmission is refused without truncating the content. Queueing is nevertheless neither proof of RF transmission nor proof of delivery.

A message returns along a learned route

When a station is actually heard over LoRa, the iGate can learn the relays that delivered its packet. If the used path contains A then B, the return path becomes B then A. Direct reception teaches a direct route.

A subsequent APRS-IS message to that recently heard station uses this route, subject to gateway settings. The first copy retained within the local window supplies the path. This policy favors the first arrival, without claiming to select the strongest signal. The route expires and is lost on restart. Without a valid route, the APRS-IS downlink does not fall back to the general beacon path.

This mechanism handles the RF return path for one iGate. It does not select a unique transmitter when several iGates can reach the same station. That coordination remains a separate problem.

Acknowledging a copy without repeating its processing

If an ACK is lost, a later copy of the message can legitimately require another ACK. Local fixes distinguish that acknowledgment from query execution: they allow another ACK without generating a second response within the relevant window. They also preserve reply-ack and make the identity actually addressed send the reply.

Local RF replies can use their fallback path when no usable route is known. This rule belongs to a different processing path from the APRS-IS downlink described above.

Remote control adds a separate check: a command must match the configured controller and carry an HMAC tag bound to its target, counter and content. Persistent counter state prevents re-execution. This authenticates those commands without encrypting their content. Ordinary APRS traffic does not acquire that authentication.

6. Station telemetry complements RXT

RXT answers a question about one particular reception. APRS station telemetry describes voltages and aggregate activity. Rates are calculated from counters and the actual elapsed time between reports, then expressed in packets per hour.

`RX_rate` counts receptions accepted after blacklist filtering. `RelRate` counts packets accepted and queued by the digi. `DrpRate` counts rejections under application rules. They do not form a delivery balance: a packet received by an iGate with relaying disabled can increase RX_rate without producing either a relay or a digi rejection. A blacklisted packet can increase DrpRate without increasing RX_rate.

These indicators describe site activity through named measurements and units. On their own, they cannot establish RF loss or successful delivery. The cause of the rate spike reported by Jon is outside the scope of this document.

7. The experimental native radio format

Direct TNC2 between software and the native radio format are independent mechanisms. The first remains textual. The second replaces the text radio envelope with compact binary encoding that must reconstruct the exact TNC2 packet.

The codec compacts supported addresses and positions. It can Huffman-encode the remaining text when that reduces its size. It checks its own result by decoding it. Information without specialized encoding can remain raw inside the native envelope. If the frame cannot be encoded, transmission falls back to text.

The transmit setting offers text by default, native, or text followed by native for trials. The last mode produces two copies. Receivers must understand the selected format. A device without a native codec does not automatically benefit from that coexistence.

Native RXT attaches measurements to precise positions in the path, avoiding text fingerprints for that association. Since the 13 October fix, this indexed context remains separate from the text during reception, queueing and relaying. When the path changes, measurements follow full callsigns. A later text output nevertheless returns to fingerprint-attribution limitations.

The intended benefit is lower size and airtime when the packet is suitable. This transport does not itself increase receiver sensitivity or site coverage. Its format remains experimental.

8. What has been verified and where to read the code

The September reports document the Mic-E and extended-identity paths into Graywolf, as well as message exchanges. The additional 28 September trial establishes delivery to a terminal through two physical relays, with the ACK returning directly to the iGate. Route learning uses a synthetic packet, which limits the conclusion.

The 13 October trials check bytes actually retransmitted over RF. They show that native RXT association survives path rewriting and queued delay, including two names sharing the same text fingerprint. They establish neither site coverage nor every possible path and radio profile.

To follow the implementation, start in the main loop of `src/LoRa_APRS_iGate.cpp`. `src/lora_utils.cpp` reads the radio, captures context and prepares transmission. `src/digi_utils.cpp` makes the relay decision. `src/station_utils.cpp` manages the queue and learned routes, among other functions. `src/rxt_protocol.cpp` handles text RXT and `src/native_aprs.cpp` the native codec. JSON and TNC outputs are in the `aprs_json_utils` and `tnc_utils` modules.

Technical descriptions and hardware evidence are separate references:

- `docs/RXT.md`: measurements, transport, interfaces and station telemetry.
- `docs/RXT_V2.md`: motivation and text v2 tuple attribution, to be read alongside the current transport described in RXT.md.
- `docs/REVERSE_MESSAGE_ROUTING.md`: return-path learning and use.
- `docs/RF_FIX_VALIDATION_20260928_en.md`: path, reply and message tests.
- `docs/LORA_APRS_NATIVE_FRAME_DRAFT0.md` and `docs/VALIDATION_RXT_NATIVE_RF_20261013.md`: native codec and RXT trials.
- In Graywolf, `docs/wiki/rxt-telemetry.md` and `docs/wiki/lora-tnc2-transceiver.md`: JSON reception, identity model and TNC2 exchanges.